

FINAL TASK: E-COMMERCE SHOPPING FLOW

Feature : End-to-end shopping flow — user registration through order placement

App: <https://automationexercise.com>

API base: <https://automationexercise.com/api>

OVERVIEW

You will implement a full test suite for a real e-commerce practice application, applying every technique covered during the course.

Repository Setup

- Create a brand-new public GitHub repository for this task.
- Do NOT fork, copy, or reuse the course repository in any way.
- Work directly on the main branch — no feature branches required.

Steps:

1. Create a new repository on GitHub
 - a. Name: name-surname-final-task
 - b. Visibility : Public
2. Clone your new repository locally
3. Set up a fresh Playwright project from scratch
4. Push your commits directly to main
5. When done, share the repository link for review (submit as .txt file on Moodle)

Deliverables

- Page Objects `pages/automationExercise/`
- Fixtures `fixtures/authenticatedShopPage.ts`
- API helper `utils/shopApiClient.ts`
- Test file `tests/finalTask.spec.ts`

- Allure tags applied to every test() call (@epic, @feature, @story, @severity)

TEST CASES

TC-SHOP-001 – Happy path: full shopping flow (register -> browse -> checkout)

Description: A new user can register, add a product to the cart, and complete a full checkout – verifying the order confirmation at the end.

Preconditions: No existing account for the generated email

Steps:

1. Navigate to / (home page)
2. Click "Signup / Login" in the navigation
3. In the New User Signup form, enter a unique name and email, then click Signup
4. Fill in the account creation form (title, name, email, password, date of birth, address fields, country, mobile number) and click Create Account
5. Verify the "Account Created!" confirmation page is shown, then click Continue
6. Confirm the navbar shows "Logged in as <username>"
7. Navigate to the Products page
8. Hover over the first product card and click Add to cart
9. In the modal, click View Cart
10. On the cart page, click Proceed To Checkout
11. On the checkout page, verify the delivery address matches the address entered during registration, then click Place Order
12. Fill in card details (name, number, CVC, expiry) and click Pay and Confirm Order

Expected result:

- "Order Placed!" confirmation message is displayed
- URL contains /payment_done/
- The order ID is visible on the confirmation page

Priority: P1

Allure: @epic("Shopping") @feature("Checkout") @story("Full E2E flow")
@severity("critical")

TC-SHOP-002 – Search: keyword search returns only matching products

Description: Searching for a product keyword filters the product list to matching items only.

Preconditions: None

Steps:

1. Navigate to the Products page (/products)
2. Enter "dress" in the search input field
3. Click the Submit (search) button

Expected result:

- The heading "Searched Products" is displayed
- All visible product cards contain the word "dress" (case-insensitive) in their name
- At least one product result is shown

Priority: P1

Allure: @epic("Shopping") @feature("Product Search") @story("Keyword search") @severity("normal")

TC-SHOP-003 – Cart: adding multiple products updates the item count

Description: Adding two different products to the cart results in both items appearing in the cart with correct individual quantities.

Preconditions: None (guest session is sufficient)

Steps:

1. Navigate to the Products page
2. Hover over the first product and click Add to cart; in the modal click Continue Shopping
3. Hover over the second product and click Add to cart; in the modal click Continue Shopping
4. View Cart

Expected result:

- The cart page shows exactly 2 rows in the product table
- Each row displays the correct product name, unit price, quantity (1), and line total
- The overall cart total equals the sum of both product prices

Priority: P1

Allure: @epic("Shopping") @feature("Cart") @story("Add multiple products")
@severity("normal")

TC-SHOP-004 – Cart: removing a product updates the cart

Description: Deleting an item from the cart removes it from the product table; if the cart becomes empty the empty-cart message is shown.

Preconditions: At least one product is already in the cart (seed via UI or API)

Steps:

1. Navigate to /view_cart

2. Click the x (delete) icon in the row of the product to remove
3. Observe the cart state

Expected result:

- The deleted product row is no longer present in the cart table
- If the cart is now empty, the text "Cart is empty!" (or equivalent) is visible
- Page does not navigate away or throw an error

Priority: P1

Allure : @epic("Shopping") @feature("Cart") @story("Remove product")
@severity("normal")

TC-SHOP-005 — Product detail: product information page shows correct data

Description: Opening a product's detail page displays the name, category, price, availability, condition, and brand fields with non-empty values.

Preconditions: None

Steps:

1. Navigate to the Products page
2. Click "View Product" for any product in the list
3. Inspect the product detail section

Expected result:

- Product name heading is visible and non-empty
- Category, price, availability, condition, and brand fields are all present and non-empty
- The Add to cart button is present

Priority: P2

Allure: @epic("Shopping") @feature("Product Detail") @story("View product info") @severity("minor")

TC-SHOP-006 – API: GET /api/productsList returns a valid product list

Description: The products API endpoint responds with a 200 response code and a non-empty array of product objects that each have the required fields.

Preconditions: None

Steps:

1. Send GET /api/productsList
2. Parse the JSON response

Expected result:

- responseCode equals 200
- products is an array with at least one element
- Each product object has: id, name, price, brand, category
- All id values are unique within the list

Priority: P1

Allure: @epic("API") @feature("Products API") @story("List all products") @severity("critical")

TC-SHOP-007 – API: POST /api/searchProduct returns matching results

Description: Searching via the API with a valid keyword returns products whose names contain that keyword.

Preconditions: None

Steps:

1. Send POST /api/searchProduct with body search_product=top

2. Parse the JSON response

Expected result:

- responseCode equals 200
- products array is non-empty
- Every product name in the response contains "top" (case-insensitive)

Priority: P1

Allure : @epic("API") @feature("Products API") @story("Search products")
@severity("normal")

TC-SHOP-008 — API: POST /api/searchProduct with missing parameter returns 400

Description: Calling the search endpoint without the required search_product parameter returns a 400 error response.

Preconditions: None

Steps:

1. Send POST /api/searchProduct with an empty body (no search_product field)
2. Parse the JSON response

Expected result:

- responseCode equals 400
- message field is present and describes the missing parameter

Priority: P2

Allure: @epic("API") @feature("Products API") @story("Missing parameter")
@severity("minor")

TC-SHOP-009 — Subscription: subscribing from the footer shows a success message

Description: A visitor can subscribe to the newsletter from the footer without being logged in and sees a confirmation.

Preconditions: None

Steps:

1. Navigate to / (home page)
2. Scroll to the Subscription section in the footer
3. Enter a valid email address in the subscription input
4. Click the subscribe arrow button

Expected result:

- The success message "You have been successfully subscribed!" is visible
- The subscription input is cleared or a visible confirmation replaces it

Priority: P2

Allure: @epic("Marketing") @feature("Newsletter") @story("Footer subscription")v@severity("minor")

TC-SHOP-010 — Session: authenticated user is redirected away from the login page

Description: A logged-in user who navigates directly to /login is redirected to the home page, preventing double-login.

Preconditions: User is logged in (use the authenticatedShopPage fixture)

Steps:

1. With an authenticated page, navigate directly to /login

2. Wait for navigation to settle

Expected result:

- The final URL is / (home page), not /login
- The navbar shows "Logged in as <username>"

Priority: P2

Allure: @epic("Auth") @feature("Session") @story("Redirect logged-in user")
@severity("minor")

IMPLEMENTATION REQUIREMENTS

1. PAGE OBJECT MODEL

Create the following classes under pages/automationExercise/:

- ShopHomePage Navigation links, subscription footer, navbar username
- SignupLoginPage New-user signup form + login form
- AccountCreationPage Full registration form, Create Account submit
- ProductsPage Product grid, hover-add-to-cart, search input
- ProductDetailPage Name, price, category, brand fields, add-to-cart button
- CartPage Product rows, quantity, total, delete icon, Proceed To
- CheckoutPage Address summary, order comment, Place Order button
- PaymentPage Card detail inputs, Pay and Confirm Order button
- PaymentConfirmationPage Order success message assertion

Each class must:

- Accept a Page in the constructor and define locators as properties
- Expose action methods and assertOnPage() / assertion helpers
- Extend a shared BaseShopPage (or re-use the existing BasePage)

2. AUTHENTICATED FIXTURE

Create fixtures/authenticatedShopPage.ts that:

- Generates a unique user with generateUser()
- Registers the user via the UI (completing the full signup form) before each test
- Exposes the authenticated Page as authenticatedShopPage
- Cleans up (deletes the account via DELETE /api/deleteAccount) in a test.afterEach hook

3. API HELPER

Create utils/shopApiClient.ts with typed methods wrapping APIRequestContext:

```
class ShopApiClient {  
  getProducts(): Promise<ProductsResponse>  
  searchProducts(keyword: string) : Promise<ProductsResponse>  
  createAccount(user: ShopUser): Promise<void>  
  deleteAccount(email: string, password: string): Promise<void>  
  verifyLogin(email: string, password: string): Promise<boolean>  
}
```

4. WAIT STRATEGIES

Apply the most appropriate strategy per interaction:

Product grid loading after navigation /api/productsList	waitForResponse on
Modal appearing after "Add to cart"	toBeVisible() on the modal element
Cart total recalculating after delete	waitForFunction / polling assertion
Page redirect after login	page.waitForURL()
Footer subscription confirmation	toBeVisible() on the success alert

5. ALLURE REPORTING

Every test() must carry the four Allure decorators shown in each test case above.

6. PLAYWRIGHT CONFIGURATION

Add a new project entry in playwright.config.ts:

```
{
  name: 'automation-exercise',
  use: { baseURL: 'https://automationexercise.com' },
  testMatch: '**/finalTask.spec.ts'
}
```

7. CI/CD

Create `github/workflows/playwright.yml` to run the new project.

GRADING CRITERIA

Criterion	Weight
All 10 test cases implemented and passing locally	30 %
POM classes with locators as properties and action/assertion methods	14 %
authenticatedShopPage fixture used for TC-SHOP-010 (and optionally others)	8 %
ShopApiClient used for TC-SHOP-006, TC-SHOP-007, TC-SHOP-008	8 %
Correct wait strategy applied (no hard waitForTimeout calls)	8 %
Allure decorators on every test	3 %
No hardcoded credentials — generateUser() used for all dynamic data	3 %

CI/CD – automation-exercise in matrix, all tests pass, Allure artifact uploaded	16 %
TOTAL	80 %